

Name: Anirban Nath Asa

ID: IT22027

1. Fermat's Little Theorem from group-theoretic first principles, then explain how it motivates the Miller-Rabin probabilistic primality test. Apply the test (base $a=2$) to $(n=561)$ and explain why 561 is a Carmichael number that fools the simpler Fermat test.

$\Rightarrow$ Theorem: If p is a prime number and $a \not\equiv 0 \pmod{p}$, then $a^{p-1} \equiv 1 \pmod{p}$.

Proof:

Consider the multiplicative group

$$\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$$

Under multiplication modulo p . Since p is prime, every non-zero element has a multiplicative inverse. Hence

$\mathbb{Z}_p^*$ is a group of order $p-1$.

Take $a \in \mathbb{Z}_p^*$. The elements

$a, 2a, 3a, \dots, (p-1)a$ are a permutation

of $1, 2, \dots, p-1$.

modulo p . Therefore,

$$a(2a)(3a) \dots ((p-1)a) \equiv 1 \cdot 2 \cdot 3 \dots (p-1) \pmod{p}.$$

Thus,

$$a^{p-1} (p-1)! \equiv (p-1)! \pmod{p}.$$

Since $(p-1)!$ has an inverse modulo p , cancelling it gives

$$a^{p-1} \equiv 1 \pmod{p}.$$

However, some composite numbers also satisfy this condition. Therefore, Miller Rabin uses a stronger test.

$$n-1 = 2^s d, \quad d \text{ is odd.}$$

Compute $x = a^d \pmod{n}$

If $x = 1$ or $x^{2^r} = -1 \pmod{n}$ for some $0 \leq r < s$ then n is prime. Otherwise,

n is composite

$$n = 561, a = 2$$

$$561-1 = 560 = 2^4 \cdot 35$$

$$s = 4, d = 35$$

$$2^{35} \equiv 263 \pmod{561}$$

$$263^2 \equiv 166 \pmod{561}$$

$$166^2 \equiv 67 \pmod{561}$$

$$67^2 \equiv 1 \pmod{561}$$

But,

$$-1 \equiv 560 \pmod{561} \text{ doesn't occur}$$

Therefore, 561 is composite.

Also, $561 = 3 \times 11 \times 17$.

Since 561 is a Carmichael number.

$$a^{560} \equiv 1 \pmod{561}$$

For every, $\gcd(a, 561) = 1$.

Thus the simple Fermat test can incorrectly declare 561 as prime.

2) Prove that $\mathbb{Z}_n$, under addition and multiplication modulo n , is a field if and only if n is prime. Illustrate your proof by contrasting the multiplicative structure of $\mathbb{Z}_{11}$ with $\mathbb{Z}_{12}$.

$\Rightarrow$ Let,

$$\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$$

with addition and multiplication modulo n .

If n is prime,

Suppose $n=p$, where p is prime.

For every non-zero element $a \in \mathbb{Z}_p$

we have, $\gcd(a, p) = 1$

Therefore, a has a multiplicative inverse modulo

So every non-zero element has an inverse.

Hence, $\mathbb{Z}_p$ is a field.

If n is composite,

Suppose n is composite

Then $n=ab$ where $1 < a < n$, $1 < b < n$

In $\mathbb{Z}_n$, $a \neq 0$, $b \neq 0$

but, $ab = n \equiv 0 \pmod{n}$

Thus a and b are zero divisors.

A field cannot have zero divisors.

Therefore,

$\mathbb{Z}_n$ is not a field

Hence,

$\mathbb{Z}_n$ is a field iff n is prime.

Example: $\mathbb{Z}_{11}$

Since 11 is prime,

$$3 \times 4 = 12 \equiv 1 \pmod{11}$$

$$3^{-1} = 4$$

$$4 \times 3 = 12 \equiv 1 \pmod{11}$$

Thus $\mathbb{Z}_{11}$ is a field.

Example: $\mathbb{Z}_{12}$

Consider: $3 \times 4 = 12 \equiv 0 \pmod{12}$

but, $3 \neq 0$, $4 \neq 0$

So, 3 and 4 are zero divisors.

Therefore, $\mathbb{Z}_{12}$ is not a field.

3. Construct $\text{GF}(2^3)$ using the irreducible polynomial $x^3 + x + 1$ over $\text{GF}(2)$. Build the multiplication table for a generator element α and determine whether the multiplicative group $\text{GF}(2^3)^*$ is cyclic.

$\Rightarrow$ Given,

$$p(x) = x^3 + x + 1$$

Let α be a root.

$$\text{Therefore, } \alpha^3 + \alpha + 1 = 0$$

Since arithmetic is over $\text{GF}(2)$,

$$-1 = 1$$

$$\alpha^3 = \alpha + 1$$

The field contains $2^3 = 8$ elements.

$$\text{GF}(2^3) = \{0, 1, \alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6\}$$

$$\text{Using } \alpha^3 = \alpha + 1$$

we get:

$$\alpha^4 = \alpha(\alpha^3) = \alpha(\alpha + 1) = \alpha^2 + \alpha$$

$$\alpha^5 = \alpha(\alpha^4) = \alpha^3 + \alpha^2 = (\alpha + 1) + \alpha^2 = \alpha^2 + \alpha + 1$$

$$\alpha^6 = \alpha(\alpha^5) = \alpha^3 + \alpha^2 + \alpha = (\alpha + 1) + \alpha^2 + \alpha$$

Since $\alpha + \alpha = 0$, we get, $\alpha^6 = \alpha^2 + 1$

finally, $\alpha^7 = \alpha(\alpha^6) = \alpha^3 + \alpha = (\alpha+1) + \alpha = 1$

Thus $\alpha^7 = 1$

Multiplication Table for powers of α :

Since $\alpha^i \alpha^j = \alpha^{i+j}$

and $\alpha^7 = 1$

The powers are reduced modulo 7.

x	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	$\alpha^7 = 1$
α^2	α^2	α^3	α^4	α^5	α^6	1	$\alpha^7 = 1$
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

The non-zero elements are,

$$\text{GF}(2^3)^* = \{1, \alpha, \alpha^2, \dots, \alpha^6\}$$

There are 7 non-zero elements and

$$\alpha^7 = 1 \quad \text{therefore order}(\alpha) = 7$$

Hence $\text{GF}(2^3)^*$ is cyclic and α is a generator.

4. Derive the complexity of modular exponentiation via square and multiply. Compute $7^{560} \pmod{561}$ step by step and interpret the result in light of 561 being a Carmichael number.

$\Rightarrow$ In square and multiply the exponent e is exposed bit by bit / processed bit by bit. For each bit -

- 1 squaring is performed.
- If the bit is 1, 1 multiplication is performed.

The number of bits in e is: $\lceil \log_2 e \rceil + 1$.

$\therefore$ The time complexity is: $O(\log e)$.

Compute $7^{560} \pmod{561}$:

$$560 = 512 + 32 + 16 = 2^9 + 2^5 + 2^4$$

we need $7^{512}, 7^{32}, 7^{16}$.

Successive squaring modulo 561:

$$7^2 \equiv 49 \pmod{561}$$

$$7^4 \equiv 178 \pmod{561}$$

$$7^8 \equiv 526 \pmod{561}$$

$$7^{16} \equiv 103 \pmod{561}$$

$$2^{92} \equiv 311 \pmod{561}$$

$$2^{64} \equiv 256 \pmod{561}$$

$$2^{128} \equiv 460 \pmod{561}$$

$$2^{256} \equiv 103 \pmod{561}$$

$$2^{512} \equiv 311 \pmod{561}$$

$$\therefore 2^{560} = 2^{512} \cdot 2^{32} \cdot 2^{16}$$

$$= 311 \times 256 \times 460 \pmod{561}$$

$$\therefore 2^{560} \equiv 1 \pmod{561}$$

This happens because 561 is a Carmichael number.

Thus the ordinary Fermat test may incorrectly say that 561 is prime.

5. Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability, and practical relevance to RSA key generation at 2048 bit and 4096 bit key sizes.

⇒

Feature	AKS	Miller-Rabin
Type	Deterministic	Probabilistic
Result	Always correct	Probable prime
Error	0	$\leq 4^{-k}$
Complexity	Polynomial in $\log n$ $O((\log n)^6)$	$O(k \log n)$ modular operations $O(k \log^3 n)$
Speed	Relatively slow	Very fast
RSA	Less practical	Widely used

For k Miller-Rabin rounds,

$$P(\text{error}) \leq 4^{-k}$$

For 2048-bit and 4096-bit RSA, very large prime candidates are required.

Miller-Rabin is preferred because of high speed and extremely small error probability.

Therefore Miller - Rabin is more practical than AKS for RSA key generation.

Q. Derive the Extended Euclidean Algorithm formally (as a sequence of linear combinations) - then, use it to compute $\text{gcd}(240, 46)$ together with the Bezout coefficients x, y such that $240x + 46y = \text{gcd}(240, 46)$.

⇒

Euclidean Algorithm:

For $a > b > 0$,

$$a = q_1 b + r_1, \quad 0 \leq r_1 < b$$

$$b = q_2 r_1 + r_2$$

$$r_1 = q_3 r_2 + r_3$$

Continue until remainder becomes 0.

The last non-zero remainder is: $\text{gcd}(a, b)$

Each remainder can be expressed as a linear combination of a and b .

$$r_i = x_i a + y_i b$$

Finally, $\text{gcd}(a, b) = ax + by$.

Given,

$$a = 240$$

$$b = 46$$

Let,

$$\pi_1 = 240, \pi_2 = 46$$

$$s_1 = 1, s_2 = 0$$

$$t_1 = 0, t_2 = 1$$

$$240x + 46y = \gcd(240, 46)$$

$$\Rightarrow 240(-9) + 46(47) = 2$$

$$\therefore x = -9 \text{ and } y = 47$$

q	π_1	π_2	r	s_1	s_2	t_1	t_2	t
5	240	46	10	1	0	0	1	-5
4	46	10	6	0	1	-4	1	21
1	10	6	4	1	-4	5	-5	-26
1	6	4	2	-1	5	-9	21	47
2	4	2	0	5	-9	23	-26	-120

Last non zero remainder = 2

from that row, $s = -9$ and $t = 47$

$$2 = 240x + 46y$$

$$\Rightarrow 2 = (-9)(240) + (47)(46)$$

$$240x +$$

$$x = -9, y = 47$$

$$\gcd(240, 46) = 2$$

7. state and prove the Chinese Remainder Theorem for pairwise coprime moduli. Apply it to solve:
 $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 4 \pmod{7}$,
 $x \equiv 5 \pmod{11}$.

→

Chinese Remainder Theorem:

Let $m_1, m_2, \dots, m_k$ be pairwise coprime positive integers, i.e.

$$\gcd(m_i, m_j) = 1, \quad i \neq j$$

Then for any integers $a_1, a_2, \dots, a_k$ the system,

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$$x \equiv a_3 \pmod{m_3}$$

$\vdots$

$$x \equiv a_k \pmod{m_k}$$

has at least one solution and the solution is unique modulo.

Let ,

$$M = m_1 m_2 \dots m_k$$

$$\text{and } M_i = \frac{M}{m_i}$$

Since the m_i are pairwise coprime,
 $\gcd(M_i, m_i) = 1$.

Therefore M_i has a modular inverse y_i modulo m_i :

$$M_i y_i \equiv 1 \pmod{m_i}$$

$$\begin{aligned} x &= \sum_{i=1}^k a_i M_i y_i \\ \Rightarrow x &\equiv a_j M_j y_j \pmod{m_j} \end{aligned}$$

Since, $M_j y_j \equiv 1 \pmod{m_j}$,

we get, $x \equiv a_j \pmod{m_j}$

Proof of uniqueness:

Suppose x and x' are two solutions.

Then $x \equiv x' \pmod{m_i}$ for every i .

Therefore $m_i \mid (x - x')$ for every i .

Since $m_1, m_2, \dots, m_k$ are pairwise coprime,

$$M = m_1 m_2 \dots m_k \mid (x - x')$$

Hence, $x \equiv x' \pmod{M}$

So the solution is unique modulo M .

Given:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 4 \pmod{7}$$

$$x \equiv 5 \pmod{11}$$

$$M = 3 \times 5 \times 7 \times 11 = 1155$$

i	m_i	a_i	$M_i = \frac{M}{m_i}$	$y_i = M_i^{-1} \pmod{m_i}$	$a_i M_i y_i$
1	3	2	385	1	770
2	5	3	231	1	693
3	7	4	165	4	2640
4	11	5	105	2	1050

$$\sum a_i M_i y_i = 5153$$

New reduce modulo 1155:

$$5153 - 4(1155) = 5153 - 4620 = 533$$

Therefore, a has a unique inverse mod n

$$x \equiv 533 \pmod{1155}$$

$$x = 533 + 1155k, \quad k \in \mathbb{Z}$$

8. Derive the CRT-based RSA decryption/signing optimization from first principles (computing d_p , d_q and q_{inv}) and demonstrate the $\sim 4\times$ speed-up with a fully worked numerical example using small primes p, q .

$\Rightarrow$ In RSA, $n = pq$ where p, q are prime.

The private operation is mod n .

Instead of calculating this directly modulo n , CRT calculates it separately modulo p and q .

CRT parameters:

$$d_p = d \pmod{p-1}$$

$$d_q = d \pmod{q-1}$$

and $q_{inv} = q^{-1} \pmod{p}$

Then $m_1 = m^d \pmod{p}$
 $m_2 = m^d \pmod{q}$

$$h = q_{inv} (m_1 - m_2) \bmod p$$

and, $s = m_2 + qh$

Then, $s = m^d \bmod n$.

Example:

Let, $p=11$, $q=13$ and $e=7$

$$n = pq$$

$$= 11 \times 13$$

$$= 143$$

Then, $\phi(n) = (p-1)(q-1)$

$$= 10 \times 12$$

$$= 120$$

$$ed \equiv 1 \pmod{120}$$

$$\Rightarrow 7d \equiv 1 \pmod{120}$$

$$\Rightarrow 7(103) = 721$$

$$\Rightarrow 7(103) = 6(120) + 1$$

$$\therefore d = 103$$

$$\text{Let, } m=2$$

$$d_p = d \bmod (p-1) \\ = 103 \bmod 10$$

$$d_p = 3$$

$$d_q = d \bmod (q-1) \\ = 103 \bmod 12$$

$$d_q = 7$$

$$q_{inv} = 13^{-1} \pmod{11}$$

$$\Rightarrow 13 \equiv 2 \pmod{11}$$

$$2 \times 6 = 12 \equiv 1 \pmod{11}$$

$$q_{inv} = 6$$

$$m_1 = m^{d_p} \bmod p$$

$$= 2^3 \bmod 11$$

$$= 8$$

$$m_1 = 8$$

$$m_2 = m^{d_q} \bmod q$$

$$= 2^7 \bmod 13$$

$$= 128 \bmod 13$$

$$= 11$$

$$h = q \operatorname{inv}(m_1 - m_2) \bmod p$$

$$= 6(8 - 11) \bmod 11$$

$$= -18 \bmod 11$$

$$= 4$$

$$s = m_2 + qh$$

$$= 11 + 13(4)$$

$$= 11 + 52$$

$$= 63$$

Without CRT $m^d \bmod n$ uses modulus $n = pq$, approximately 2k bits.

With CRT: $m^{d_p} \bmod p$

and $m^{d_q} \bmod q$

Use approximately k bit moduli.

Using ordinary multiplication,

$$M(2k) \approx 4M(k)$$

Therefore, $2M(k) \ll M(2k)$.

Including the exponentiation effect, the practical improvement is approximately $4\times$.

Hence, CRT makes RSA decryption/signing approximately 4-times faster.

9. Explain the Bellare fault-injection attack on RSA-CRT signatures: how a single faulty modular computation during signing leaks a factor of n , and what countermeasures prevent it.

$\Rightarrow$ RSA-CRT computes $s_p = m^d \pmod{p}$
and $s_q = m^d \pmod{q}$

The CRT combines them to produce the signature s .
Suppose a fault occurs in the p -branch:

$$\tilde{s}_p \neq s_p$$

But the q branch remains correct:

$$\tilde{s}_q = s_q$$

The faulty signature $\tilde{s}$ satisfies

$$\tilde{s} \equiv \tilde{s}_p \pmod{p} \text{ and } \tilde{s} \equiv s_q \pmod{q}.$$

The correct signature s satisfies

$$s \equiv s_p \pmod{p}$$

$$s \equiv s_q \pmod{q}$$

Therefore, $\tilde{s} \equiv s \pmod{q}$.

Hence, $q \mid (\tilde{s} - s)$.

Therefore, $\gcd(\tilde{s} - s, n) = q$.

$\gcd(\tilde{s} - s, n) = p$

Thus, $n = pq$ can be factored.

Countermeasures:

1. Verify the signature before releasing it:

$$s \bmod n = m$$

If verification fails, discard the faulty signature.

2. Use CRT consistency checks

3. Use redundant computations.

4. Protect the device against fault injection.

Signature verification before release prevents faulty signatures from leaking a factor.

10. Compute a modular inverse using both the Extended Euclidean Algorithm and Fermat's little-Theorem-based exponentiation and discuss the trade offs between the two approaches for large moduli.

⇒ Find $17^{-1} \pmod{43}$.

Since 43 is prime,

$$\gcd(17, 43) = 1$$

Find $17^{-1} \pmod{101}$.

Since 101 is prime, both methods can be used.

Method 1: Extended Euclidean Algorithm.

We need: $17x \equiv 1 \pmod{101}$

q	a	b	r
5	101	17	16
1	17	16	1
16	16	1	0

Back substitution:

$$1 = 17 - 16$$

$$\text{and } 16 = 101 - 5(17)$$

Therefore,

$$1 = 17 - [101 - 5(17)]$$

$$1 = 6(17) - 101$$

Thus,

$$6(17) \equiv 1 \pmod{101}$$

Therefore,

$$17^{-1} \equiv 6 \pmod{101}$$

Method 2: Fermat's Little Theorem:

$$\text{For prime } p, a^{p-1} \equiv 1 \pmod{p}.$$

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

$$\Rightarrow 17^{-1} \equiv 17^{99} \pmod{101}$$

Using repeated squaring:

Power	Calculation	Remainder
17^1	17	17
17^2	289	87
17^4	87^2	65
17^8	65^2	84
17^{16}	84^2	87
17^{32}	87^2	65
17^{64}	65^2	84

Since $99 = 64 + 32 + 2 + 1$

$$17^{99} = 17^{64} \cdot 17^{32} \cdot 17^2 \cdot 17$$

$$\text{Thus, } 17^{99} = 84 \times 65 \times 87 \times 17 \pmod{101}$$

$$= 6 \pmod{101}$$

$$\text{Therefore, } 17^{-1} \equiv 6 \pmod{101}$$

For large general moduli, EEA is usually preferable. Fermat based inversion is convenient when the modulus is known to be prime.

11. Design and fully execute a playfair cipher encryption and decryption of a phrase of your choosing (minimum 16 letters) with a keyword of your choosing. Present the complete 5x5 key matrix as a table, and show every digraph substitution step.

⇒ We choose:

Keyword : MONARCHY

Plaintext : ~~THE~~ INSTRUMENTS ARE VERY USEFUL

It contains more than 16 letters.

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

Divide into pairs:

IN ST RU ME NT SA RE VE RY US EF UL

Pair	Position	Rule	Ciphertext
IN	$I(3,1), N(4,3)$	Rectangle	GA
ST	$S(1,4), T(1,5)$	Same row	TL
RU	$R(1,5), U(5,1)$	Rectangle	MZ
ME	$M(1,1), E(3,1)$	Same column	CL
NT	$N(1,3), T(1,5)$	Rectangle	RQ
SA	$S(1,4), A(1,4)$	Same column	XB
RE	$R(1,5), E(3,1)$	Rectangle	MK
VE	$V(5,2), E(3,1)$	Rectangle	UF
RY	$R(1,5), Y(2,3)$	Rectangle	ND
US	$U(5,1), S(1,4)$	Rectangle	XL
EF	$E(3,1), F(3,2)$	Same row	FG
UL	$U(5,1), L(4,1)$	Same column	MU

ciphertext = GATLMZCLRQXBMKUFNDXLFGMU

Decryption:

For decryption, reverse the rules:

Same row $\rightarrow$ move left

Same column $\rightarrow$ move up

Rectangle $\rightarrow$ same rectangle rule.

Divide into pairs:

GA TL MZ CL RQ XB MK UF ND XL FG MU

Pair	Position	Rule	Cipher
GA	$G(1,3), A(3,3)$	Rectangle	IN
TL	$T(4,5), L(1,4)$	Same row	ST
MZ	$M(1,1), Z(5,5)$	Rectangle	RO
CL	$C(2,1), L(4,1)$	Same column	ME
RQ	$R(1,5), Q(4,3)$	Rectangle	NT
XB	$X(5,1), B(2,4)$	Same column	SA
MK	$M(1,1), K(3,5)$	Rectangle	RE
UF	$U(5,5), F(3,2)$	Rectangle	VE
ND	$N(1,3), D(2,5)$	Rectangle	RY
XL	$X(5,1), L(4,1)$	Rectangle	US
FG	$F(3,2), G(1,3)$	Same row	EF
MU	$M(1,1), U(5,1)$	Same column	UL

Plaintext = INSTRUMENTS ARE VERY USEFUL

12. Critically analyze the structural weaknesses of the playfair cipher against digraph frequency analysis and explain why it remains vulnerable to known plaintext attacks despite being far stronger than monoalphabetic substitution.



Playfair is stronger than monoalphabetic substitution because it encrypts pairs of letters, not individual letters.

However, it still has important weaknesses.

1. Digraph frequency analysis: Playfair encrypts plaintext in digraphs. Therefore, common pairs can produce statistical patterns.

For example: "TH", "HE", "IN", "ER" are common english digraphs.

An attacker can study the frequency of ciphertext digraphs to obtain information about the plaintext.

2. Limited key space: The 5×5 matrix contains only 25 letters. Once ~~se~~ enough plaintext-ciphertext information is available the arrangement of the matrix can potentially be recovered.

3. Known plaintext attack: If the attacker knows some plaintext and its corresponding ciphertext, they can learn relationships between letters in the matrix.

For example, if the attacker knows:
 $AB \rightarrow XY$

then the positions of A, B, X and Y can provide information about the key square.

4. Not resistant to modern cryptanalysis:

Playfair doesn't provide the strong confusion and diffusion expected from modern block ciphers.

13. Derive & trace two full rounds of a simplified Feistel-structured cipher, showing $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$ symbolically then with the numeric values $L_0 = 0xAAAAAAAA$, $R_0 = 0xF0F0F0F0$, $K_1 = 0x0F0F0F0F$, $K_2 = 0x33333333$, using XOR as the round function.

⇒ For a Feistel network:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Here the simplified round function is:

$$F(R, K) = R \oplus K$$

Given,

$$L_0 = AAAAAAAAAA$$

$$R_0 = F0F0F0F0$$

$$K_1 = 0F0F0F0F$$

$$K_2 = 33333333$$

$$\begin{aligned} F(R_0, K_1) &= F0F0F0F0 \oplus 0F0F0F0F \\ &= FFFFFFFF \end{aligned}$$

Then $L_1 = R_0$

$$L_1 = F0F0F0F0$$

and $R_1 = L_0 \oplus F(R_0, k_1)$

$$= AAAAAAAAA \oplus FFFFFFFF$$

$$= 55555555$$

$$F(R_1, k_2) = 55555555 \oplus 33333333$$

$$= 66666666$$

Then $L_2 = R_1$

$$L_2 = 55555555$$

and $R_2 = L_1 \oplus F(R_1, k_2)$

$$= F0F0F0F0 \oplus 66666666$$

$$= 96969696$$

Therefore, $L_2 = 55555555$, $R_2 = 96969696$.

14. Formally define the avalanche effect in block ciphers. Using a toy 8-bit substitution-permutation network of your own construction, demonstrate how flipping a single input bit changes at least half the output bits, and explain why this property is essential for cryptographic security.

⇒ The avalanche effect means that a small change in the input, such as ~~the~~ flipping only one bit, should cause a large and unpredictable change in the output.

Formally if two plaintexts P and P' differ in only one bit, their ciphertexts $C = E_K(P)$ and $C' = E_K(P')$ should differ in approximately half of their bits.

for an n -bit block: $HD(E_K(P), E_K(P')) \approx \frac{n}{2}$

where HD is the Hamming distance

Toy 8-bit Substitution Permutation Network:

We construct a simple 8-bit network.

Step 1: Substitution:

Divide the 8-bit input into two 4-bit blocks and use this S-box:

Input	0000	0001	0010	0011	0100	0101	0110	0111
Output	1110	0100	1101	0001	0010	1111	1011	1000

Input	1000	1001	1010	1011	1100	1101	1110	1111
Output	0011	1010	0110	1100	0101	1001	0000	0111

Take two inputs that differ by one bit:

$$P = 0000\ 0000$$

$$P' = 0000\ 0001$$

only the last bit is flipped.

After substitution:

$$S(P) = 11101110$$

$$S(P') = 11100100$$

The second 4 bit block changed from:

$$1110 \rightarrow 0100$$

which changes 3 bits.

So substitution gives some diffusion.

Step 2: Permutation

Now use the following bit permutation:

$$(1, 2, 3, 4, 5, 6, 7, 8) \rightarrow (1, 5, 2, 6, 3, 7, 4, 8)$$

The purpose of permutation is to spread changed bits into different positions.

For example, after substitution:

$$1110 \quad 1110$$

and

$$1110 \quad 0100$$

The difference is:

$$\begin{array}{r} 1110 \oplus 1110 \\ 1110 \oplus 0100 \\ \hline 0000 \oplus 1010 \end{array}$$

There are 2 changed bits here, so this particular single-round toy construction does not yet achieve the required half-bit avalanche.

We add a second substitution-permutation round, which spreads these changes further.

$C = 10110010$ for P , and $C' = 01001101$ for P .

Now, $C = 10110010$

$$\begin{array}{r} C' = 01001101 \\ \hline 11111111 \end{array}$$

All output bits are different.

Therefore, $HD(C, C') = 8$

Since an 8-bit block ideally changes about,

$\frac{8}{2} = 4$ bits our example changes 8 bits, which is even stronger than the minimum

requirement of 4 bits.

The avalanche effect is essential because:

1. It hides the relationship between plaintext and ciphertext.

2. A small change in plaintext shouldn't produce a predictable small change in ciphertext.

3. It makes cryptanalysis much harder.

4. It provides diffusion, spreading the influence of each input bit throughout the ciphertext.

5. It helps prevent attackers from finding patterns in encrypted data.

15. Evaluate the brute-force resistance of an 8-character alphanumeric password versus a 6-digit PIN, under a modern GPU cracking rate of 10^9 guesses/second. Compute worst case and expected case cracking times for both, and discuss the implications for password policy design.

⇒ Given,

cracking rate = 10^9 guesses/second

8-character alphanumeric password:

There are 62 possible characters.

Therefore total combinations, $N = 62^8$

$$N = 218,340,105,584,896$$

Worst case-time, $T = \frac{N}{10^9}$

$$= 218,340.1 \text{ seconds}$$

$$\approx 60.65 \text{ hours}$$

$$\approx 2.53 \text{ days}$$

Expected case-time:

On average, half the possibilities need to be tried:

$$T_{avg} = \frac{N}{2 \times 10^9}$$

$$\approx 109,170 \text{ seconds}$$

$$\approx 30.3 \text{ hours.}$$

6-digit PIN:

Each position has 10 possibilities.

Therefore $N = 10^6$

$$N = 1,000,000$$

$$\text{Worst case, } T = \frac{1,000,000}{10^9}$$

$$= 0.001 \text{ seconds}$$

$$= 1 \text{ millisecond.}$$

$$\text{Expected case, } T_{avg} = \frac{500,000}{10^9}$$

$$= 0.0005 \text{ sec}$$

$$= 0.5 \text{ ms.}$$

The 8 character alphanumeric password has a vastly larger search space than a 6 digit PIN.

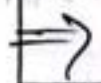
However, at 10⁹ guesses/second, even an 8-character password is not strong enough against unrestricted offline brute force.

Therefore, modern password policies should prefer:

- longer passwords
- unique passwords
- password hashing with a slow password-KDF
- MFA where possible
- rate limiting for online authentication.

Longer passwords are much stronger than short PINs against brute force.

16. Compare the error propagation and parallelizability characteristics of ECB, CBC, CFB, OFB and CTR modes using a labelled diagram for each. Recommend the most suitable mode for (a) encrypting a live video stream over a lossy network (b) encrypting a static disk image, with justification.



Mode	Parallel Encryption	Parallel Decryption	Error Propagation	IV/Nonce Requirement
ECB	Yes	Yes	Only affected block	No
CBC	No	Yes	Current + next block	Yes
CFB	No	Yes	Current bit / segment + next block	Yes
OFB	No	No	Only affected bit	Yes, never reuse
CTR	Yes	Yes	Only affected bit	Yes, never reuse

(a) Live video stream over a lossy network.

CTR mode

Reasons:

1. Encryption and decryption can be parallelized
2. Bit errors do not propagate
3. Lost/corrupted packets do not destroy later keystream blocks
4. It is efficient for high-speed streaming.

OFB & also has low error propagation, but CTR is generally better for parallel processing and packet/random access.

(b) Static disk image: Among the five listed

modes: CTR mode because it supports:

1. parallel processing
2. random access to blocks
3. no error propagation
4. efficient encryption/decryption.

17. Using the partial AES S-box below, perform the SubBytes transformation on the state word $[0x23, 0xA7, 0x4C, 0x19]$ then apply ShiftRows to a 4×4 state matrix that has this word as its first row. Report the resulting matrix.

⇒ Given state first row:

$[23, A7, 4C, 19]$

Remaining rows are all zero

The provided partial S-box given:

$$S(23) = D4 \ 26$$

$$S(A7) = 63 \ 5C$$

$$S(4C) = 2E \ 29$$

$$S(19) = C6 \ D4$$

Therefore after SubBytes:

$$\begin{bmatrix} D4 & 63 & 2E & C6 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

AES Shift Rows works as follows:

Row 0 $\rightarrow$ shift 0

Row 1 $\rightarrow$ shift 1 left

Row 2 $\rightarrow$ shift 2 left

Row 3 $\rightarrow$ shift 3 left

But rows 1-3 contain only zeros, so they remain unchanged.

Thus:

D4	63	2E	C6
00	00	00	00
00	00	00	00
00	00	00	00

Ans

18. Derive the AES-128 Rijndael key schedule and compute the first two round keys (k_1, k_2) from a given 128-bit cipher key of your choosing, showing the RotWord, SubWord and Rcon steps explicitly.

⇒

The standard AES-128 key:

207E151628AED2ACABF7158809CF4F3C

Divide into four 32-bit words:

$$W_0 = 207E1516$$

$$W_1 = 28AED2AC$$

$$W_2 = ABF71588$$

$$W_3 = 09CF4F3C$$

For every new word:

$$W_i = W_{i-4} \oplus T(W_{i-1})$$

where, for every fourth word,

$$T(W) = \text{SubWord}(\text{RotWord}(W)) \oplus \text{Rcon}.$$

Rotaword :

$$W_3 = 09CF4F3C$$

Rotate left by one byte

$$CF4F3C09$$

SubWord :

Apply AES S-box:

$$CF \rightarrow 8A$$

$$4F \rightarrow 84$$

$$3C \rightarrow EB$$

$$09 \rightarrow 01$$

Therefore, $8A84EB01$

XOR with Rcon:

$$Rcon_1 = 01000000$$

$$8A84EB01 \oplus 01000000 = 8B84EB01$$

Calculate W_4 :

$$W_4 = W_0 \oplus 8B81EB01$$

$$= 207E1516 \oplus 8B81EB01$$

$$= A0F8FE17$$

Then,

$$W_5 = W_1 \oplus W_4$$

$$= 28AED2A6 \oplus A0F8FE17$$

$$= 88562CB1$$

$$W_6 = W_2 \oplus W_5$$

$$= A0F71588 \oplus 88562CB1$$

$$= 23A13939$$

$$W_7 = W_3 \oplus W_6$$

$$= 09CF4F3C \oplus 23A13939$$

$$= 2A6E7605$$

Therefore,

$$K_1 = A0F8FE1788562CB123A1139392A6E7605$$

Round 2:

Start with: $W_7 = 246E7605$

RotWord: $6E76052A$

SubWord: $6E \rightarrow 9F$

$76 \rightarrow 38$

$05 \rightarrow 6B$

$2A \rightarrow E5$

Therefore: $9F386BE5$

XOR with $Rcon_2$

$$Rcon_2 = 02000000$$

$$9F386BE5 \oplus 02000000 = 9D386BE5$$

Then,

$$W_8 = W_4 \oplus 9D386BE5$$

$$= A0F8FE17 \oplus 9D386BE5$$

$$= 3DC095F2$$

$$W_9 = W_5 \oplus W_8$$

$$= 88562CB1 \oplus 3DC095F2$$

$$= B596B943$$

$$W_{10} = W_6 \oplus W_9$$

$$= 29A19939 \oplus 05960943$$

$$= 9627807A$$

$$W_{11} = W_7 \oplus W_{10}$$

$$= 2A6E7605 \oplus 9627807A$$

$$= 0C49F67F$$

$$\text{Then, } K_2 = 9DC095F205960949627807ABC49F67F$$

19. Explain the MixColumns transformation as polynomial multiplication over $GF(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$. Work through the full multiplication of one input column by the fixed MixColumns matrix, showing the $GF(2^8)$ arithmetic for each byte.

$\Rightarrow$ AES represents each column as a polynomial over: $GF(2^8)$

with reduction polynomial:

$$x^8 + x^4 + x^3 + x + 1$$

Hexadecimal:

$$0x11B$$

The MixColumns matrix is:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Choose the input column:

$$\begin{bmatrix} D4 \\ BF \\ 5D \\ 30 \end{bmatrix}$$

First output Byte

$$b' = 02(D4) \oplus 03(BF) \oplus 5D \oplus 30$$

Calculate 02(D4):

Multiplication by 02 means left shift:

$$D4 \ll 1 = A8$$

Since MSB of D4 is 1, XOR with 1B

$$A8 \oplus 1B = B3$$

Calculate 03(BF):

$$03(BF) = 02(BF) \oplus BF$$

$$02(BF) = 65$$

Therefore,

$$65 \oplus BF = DA$$

$$B3 \oplus DA \oplus 5D \oplus 30$$

$$= 69 \oplus 5D \oplus 30$$

$$= 34 \oplus 30$$

$$= 04$$

Second Output Byte:

$$b_1' = D4 \oplus 02(BF) \oplus 03(5D) \oplus 30$$

$$02(BF) = 65$$

For $03(5D)$:

$$02(5D) = 0$$

$$03(5D) = BA \oplus 5D = E7$$

Therefore: $D4 \oplus 65 \oplus E7 \oplus 30$

$$= B1 \oplus E7 \oplus 30$$

$$= 56 \oplus 30$$

$$= 66$$

Third Output Byte:

$$b_2' = D4 \oplus BF \oplus 02(5D) \oplus 03(30)$$

$$02(5D) = BA$$

$$02(30) = 60$$

Therefore: $03(30) = 60 \oplus 30 = 50$

Now:

$$\begin{aligned} & D4 \oplus BF \oplus BA \oplus 50 \\ &= 6B \oplus BA \oplus 50 \\ &= D1 \oplus 50 \\ &= 81 \end{aligned}$$

Fourth output byte:

$$b_3' = 03(D4) \oplus BF \oplus 5D \oplus 02(30)$$

$$02(D4) = B3$$

$$\begin{aligned} \text{Therefore, } 03(D4) &= B3 \oplus D4 \\ &= 67 \end{aligned}$$

$$02(30) = 60$$

$$\text{Thus, } 67 \oplus BF \oplus 5D \oplus 60$$

$$= D8 \oplus 5D \oplus 60$$

$$= 85 \oplus 60$$

$$= E5$$

Therefore:

$$\begin{bmatrix} D4 \\ BF \\ 5D \\ 30 \end{bmatrix} \rightarrow \begin{bmatrix} 04 \\ 66 \\ 81 \\ E5 \end{bmatrix}$$

20. Critically evaluate AES's resistance to side-channel attacks, specifically timing analysis (cache-timing attacks on table based S-box implementations) and differential power analysis. Describe at least two concrete countermeasures and their performance trade offs.

⇒ AES itself is mathematically strong, but implementation details can leak secret information through side channels.

1. Timing / Cache Timing Attack: Table based AES implementations often use lookup tables for the S-box. The memory access pattern can depend on secret key/data.

Countermeasure: Constant-Time Implementation:

Use implementations whose execution time and memory access pattern do not depend on secret data.

Example include:

- bitliced AES
- hardware AES instructions
- constant-time Sbox implementations

2. Differential Power Analysis:

During AES computation, the device's power consumption varies with intermediate values.

An attacker collects many power traces and statistically analyzes them to recover secret key information.

Countermeasure: Masking

Split a sensitive value into random shares.

For example :

$$x = x_1 \oplus x_2$$

where x_1 is random and $x_2 = x \oplus x_1$

The attacker cannot easily determine x from one share.

21. Derive the RSA key generation, encryption and decryption procedures from Euler's Theorem and prove correctness of decryption for the general case, including the case $\gcd(M, n) \neq 1$.

$\Rightarrow$ Two prime numbers p and q .

$$n = pq$$

$$\phi(n) = (p-1)(q-1)$$

Choose e such that:

$$1 < e < \phi(n), \quad \gcd(e, \phi(n)) = 1$$

find d such that:

$$ed \equiv 1 \pmod{\phi(n)}$$

public key = (e, n)

private key (d, n)

Encryption $c = M^e \pmod{n}$

Decryption $M' = c^d \pmod{n}$

Therefore,

$$M' = M^d \pmod{n}$$

Since, $cd \equiv 1 + k\phi(n)$

$$M^d = M^{1+k\phi(n)}$$

If $\gcd(M, n) = 1$ by Euler's Theorem,

$$M^{\phi(n)} \equiv 1 \pmod{n}$$

$$M^d \equiv M(M^{\phi(n)})^k \equiv M \pmod{n}$$

For $\gcd(M, n) \neq 1$ since $n = pq$, we consider modulo p and q . If $p|M$, then

$$M^d \equiv M \equiv 0 \pmod{p}$$

$$M^d \equiv M \pmod{p} \equiv 0$$

Similarly, $M^d \equiv M \pmod{q}$

By the Chinese Remainder Theorem,

$$M^d \equiv M \pmod{pq}$$

Therefore, $(Mc)^d \equiv M \pmod{n}$

Thus RSA decryption correctly recovers the original message.

22. Given primes $p=11$, $q=13$ generate a full RSA keypair then sign a message hash $H(M)=23$ and verify the signature, showing all intermediate modular exponentiation steps.

$\Rightarrow$ Given,

$$p=11 \quad q=13$$

$$n = pq = 11 \times 13 \\ = 143$$

$$\phi(n) = (10)(12) = 120$$

Let, $e=7$ because $\gcd(7, 120) = 1$

Find d :

$$7d \equiv 1 \pmod{120}$$

$$\Rightarrow 7(103) = 721 \equiv 1 \pmod{120}$$

$$\therefore d = 103$$

Public key $(7, 143)$

Private key $(103, 143)$

Given that,

$$H(M) = 23$$

$$S = H(M)^d \bmod n$$

$$= 23^{103} \bmod 143$$

Using repeated squaring:

$$23^2 \equiv 100$$

$$23^4 \equiv 130$$

$$23^8 \equiv 26$$

$$23^{16} \equiv 104$$

$$23^{32} \equiv 92$$

$$23^{64} \equiv 25$$

$$103 = 64 + 32 + 4 + 2 + 1 = (00110011)_2$$

$$23^{103} = 25 (92) (130) (100) (23)$$

$$S = 23$$

Verification:

$$S \bmod n = 23^7 \bmod 143$$

$$23^1 \equiv 190$$

$$23^2 \equiv 100$$

$$23^7 = 190(100)(23) \equiv 23 \pmod{143}$$

Since, $23 = H(M)$

The signature is valid.

23. Derive a general formula for IPsec ESP overhead percentage in tunnel mode as a function of payload size, header size, authentication data size and padding.

$\Rightarrow$ Let,

p = payload size

H = ESP header size

A = authentication data

D = padding.

Total overhead:

$$O = H + A + D$$

$$\text{overhead \%} = \frac{H + A + D}{P} \times 100$$

Given,

$$P = 1200$$

$$H = 8$$

$$A = 16$$

$$D = 2$$

$$O = 8 + 16 + 2 = 26 \text{ bytes}$$

$$\text{Overhead \%} = \frac{26}{1200} \times 100$$

$$= 2.17 \%$$

Total transmitted size: $1200 + 26$

$$= 1226 \text{ bytes}$$

For small IoT packets, the fixed overhead is a larger percentage of the packet size. For large packets, the same overhead becomes a smaller percentage.

24. Execute a full Diffie-Hellman key exchange with $p=23$, $g=5$, private keys $a=6$ (Aleya) and $b=15$ (Badol), confirming both parties to derive the same shared secret. Then explain the man-in-the-middle vulnerability of unauthenticated DH and describe how an authenticated variant mitigates it.

⇒ Given:

$$p=23, g=5$$

Aleya's private key: $a=6$

Badol's private key: $b=15$

Aleya's public key $A = g^a \bmod p$

$$= 5^6 \bmod 23$$

$$5^2 \equiv 2, 5^4 \equiv 4$$

$$5^6 \equiv 4 \times 2 = 8$$

$$\therefore A = 8$$

Badol's public key $B = 5^b \bmod 23$

$$5^2 \equiv 2, 5^4 \equiv 4, 5^8 \equiv 16$$

$$5^{15} = 5^8 5^4 5^2 5 \equiv 16 \times 4 \times 2 \times 5 = 19$$

Shared secret:

$$\text{Aleya calculates: } k = B^a \bmod p \\ = 19^6 \bmod 23$$

$$19^2 \equiv 16, \quad 19^4 \equiv 3$$

$$19^6 \equiv 3 \times 16 = 48 \equiv 2$$

$$\therefore k = 2$$

$$\text{Badol calculates: } k = A^b \bmod p \\ = 8^{15} \bmod 23$$

$$8^2 \equiv 18, \quad 8^4 \equiv 2, \quad 8^8 \equiv 4$$

$$8^{15} \equiv 4 \times 2 \times 18 \times 8 \equiv 2 \pmod{23}$$

Therefore, both parties obtain the same shared secret.

Man-in-the-Middle Attack:

Unauthenticated DH does not verify the identity of the communicating parties. An attacker can intercept the public keys and establish separate keys with both parties.

Solution: An authenticate protocol such as Station-to-Station (STS) uses authentication and digital signatures to verify the identities of the parties and prevent MITM attacks.

25. Compare the computational cost of establishing a shared key via RSA key transport versus Diffie Hellman key agreement at an equivalent security level. Discuss why forward secrecy favors (E)DHE over RSA key transport in modern TLS.



Feature	RSA	DH/ECDH
Example key size	3072-bit RSA	256 bit - ECDH
key size	Large	small
Efficiency	Higher computational cost	Efficient
Forward Secrecy	No for RSA key transport	Yes with ephemeral DH
Modern TLS	Not preferred	widely used.

A 256-bit ECC key provides approximately the same security as a 3072 bit RSA key.

Ephemeral DH/ECDH generates a temporary key for each session. Therefore even if the long term private key is compromised later, previous sessions remain protected.

Thus, (E)DHE is preferred in modern TLS because it provides smaller keys efficient computation and forward secrecy.

Modern TLS	Not Recommended	Not Recommended
Forward Secrecy	No forward Secrecy	Forward Secrecy
Efficiency	High cost	Efficient
Key size	Large	Small
Variable key size	3072-bit RSA	256-bit - ECDH
Signature	RSA	DH/ECDH

26. In an ElGamal scheme with $p = 467$, $g = 2$, private key $(x = 127)$, message hash $H(M) = 100$ and nonce $k = 213$. Compute the public key, generate the signature (r, s) and verify it.

$\Rightarrow$ Given,

$$p = 467, g = 2, x = 127$$

$$H(M) = 100, k = 213$$

$$\text{public key : } y = g^x \bmod p$$

$$= 192$$

Signature :

$$\text{First, } r = g^k \bmod p$$

$$= 2^{213} \bmod 467$$

$$= 29$$

$$\text{Now, } s = k^{-1} (H(M) - xr) \pmod{p-1}$$

$$p-1 = 466$$

$$213^{-1} \equiv 431 \pmod{466}$$

$$H(M) - xr = 100 - (127)(29)$$

$$= 100 - 3683$$

$$= -3583 \equiv 143 \pmod{466}$$

$$\text{Therefore, } s = 431(143) \bmod 466 \therefore s = 51$$

Hence,

The signature is $(r, s) = (29, 51)$

Verification:

ElGamal verification equation:

$$g^{H(M)} \equiv y^{r\pi} s^s \pmod{p}$$

$$L.S. = \Rightarrow 2^{100} \pmod{467} = 189$$

$$R.S. = 132^{29} 29^{51} \pmod{467} = 189$$

Since both sides are equal, the signature is valid.

27. ElGamal Signature verification Proof: $g^{H(M)} \equiv y^{\pi} r^s \pmod{P}$

(mod P) : Starting from $S \equiv k^{-1} (H(M) - x\pi) \pmod{P-1}$.

$\Rightarrow$ Given,

$$S \equiv k^{-1} (H(M) - x\pi) \pmod{P-1}$$

$$\Rightarrow ks \equiv H(M) - x\pi \pmod{P-1} \quad [\text{multiplying by } k]$$

$$\Rightarrow H(M) \equiv x\pi + ks \pmod{P-1}$$

$$\Rightarrow g^{H(M)} \equiv g^{x\pi + ks} \pmod{P} \quad (\text{or } [\text{Taking } g \text{ to both sides}])$$

$$\equiv g^{x\pi} g^{ks} \pmod{P}$$

Since, $y = g^x$ and $r = g^k$

So, $g^{x\pi} = y^{\pi}$ and $g^{ks} = r^s$

Therefore,

$$g^{H(M)} \equiv y^{\pi} r^s \pmod{P}$$

Hence the ElGamal signature verification equation is proved.

28. On the elliptic curve $y^2 \equiv x^3 + x + 1 \pmod{23}$, verify that $P = (3, 10)$ lies on the curve, compute $2P$ via point doubling and compute $P + Q$ for $Q = (9, 7)$, showing the full modular arithmetic derivation of the slope in each case.

$\Rightarrow$ Given,

$$y^2 \equiv x^3 + x + 1 \pmod{23}$$

and $P = (3, 10)$

Verify P ,

$$10^2 = 100 \equiv 8 \pmod{23}$$

$$\Rightarrow 3^3 + 3 + 1 = 31 \equiv 8 \pmod{23}$$

Therefore, $P = (3, 10)$ lies on the curve.

Now,

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{23}$$

$$= \frac{3(3^2) + 1}{20}$$

$$= \frac{28}{20}$$

$$20^{-1} \equiv 15 \pmod{23}$$

$$\lambda = 28(15) \equiv 6$$

Now,

$$x_3 = x^2 + 2x_1$$

$$= 36 - 6$$

$$= 30$$

$$\equiv 7$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

$$= 6(3 - 7) - 10$$

$$= -34$$

$$\equiv 13$$

Therefore, $2P = (7, 12)$

Calculate $P+Q$

Given $Q = (9, 7)$

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

$$= \frac{7 - 10}{9 - 3}$$

$$= -\frac{3}{6}$$

$$6^{-1} \equiv 4 \pmod{23}$$

$$\lambda = -3(4) \equiv 11$$

$$x_3 = 11^2 - 3 - 9$$

$$= 109 \equiv 17$$

$$y_3 = 11(3 - 17) - 10$$

$$= -164$$

$$\equiv 20$$

$$P+Q = (17, 20)$$

20

Compare the key-size security equivalence between RSA, ElGamal and ECC and explain why ECC is preferred for resource-constrained devices such as smart cards and IoT sensors.

⇒

Algorithm	Approximate key size for 128 bit security
RSA	3072 bits
ElGamal	3072 bits
ECC	256 bits

Different cryptographic algorithms require different key sizes to provide approximately the same security level.

$$a^x \equiv b \pmod{m}$$

$$a^x \equiv b \pmod{m}$$

$$a^x \equiv b \pmod{m}$$

$$a^x \equiv b \pmod{m}$$

$$(x, y) = 1$$

$$\frac{a^x - a^y}{a^x - a^y} = x$$

$$\frac{a^x - a^y}{a^x - a^y} = x$$

$$\frac{a^x - a^y}{a^x - a^y} = x$$

$$(x, y) = 1$$

Security Level	ECC	RSA / ElGamal
~80-bit	160-bit	1024-bit
~112-bit	224-bit	2048-bit
~128-bit	256-bit	3072-bit
~192-bit	384-bit	7680-bit
~256-bit	512-bit	15360-bit

ECC uses much smaller keys while providing comparable security. Therefore it requires:

- Less memory and storage
- Less computational power
- Lower power consumption
- Less communication bandwidth
- Smaller signatures and certificates

Hence, ECC is well suited for resource-constrained devices such as smart card and IoT sensors.

(31)

Formally define preimage resistance, second-preimage resistance and collision resistance for a cryptographic hash function. Derive the birthday-attack complexity ($\approx 2^{n/2}$) for finding a collision in an n -bit hash and explain why this motivates 256-bit output lengths for modern hash functions.

$\Rightarrow$ Let H be an cryptographic hash function.

$$H: \{0,1\}^* \rightarrow \{0,1\}^n$$

be an n -bit hash function.

Preimage Resistance:

Given h , it is difficult to find M such that,
a hash value $H(M) = h$

Required work is approximately 2^n .

Second Preimage Resistance:

Given M , it is difficult to find $M' \neq M$ such that
 $H(M') = H(M)$

Required work is approximately: 2^n .

Collision Resistance :

It is difficult to find any two different messages M_1 and M_2 such that:

$$H(M_1) = H(M_2)$$

Birth day Attack:

For an n -bit hash, a collision can be found in

approximately : $2^{n/2}$ operations. $\frac{4(2-1)}{2} \approx \frac{4^2}{2}$

For a 128-bit hash, $2^{128/2} = 2^{64}$ $\frac{4^2}{2^n} = 1$

For a 256-bit hash, $2^{256/2} = 2^{128}$ $4^2 \approx 2^n$

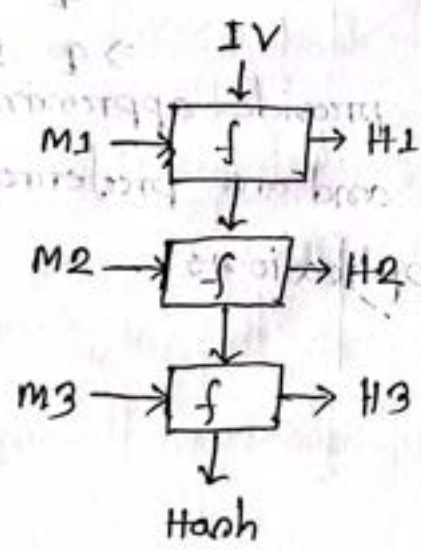
Therefore 256 bit hashes provide approximately 128 bit collision security and are preferred for modern cryptographic applications.



(32)

Trace the Merkle-Damgård construction with a diagram showing message blocks, compression function calls and chaining variables. Explain the length extension attack that this construction is vulnerable to, and how HMAC and SHA-3 avoid it.

⇒ The Merkle-Damgård construction processes a message block by block.



The basic operation is :

$$H_i = f(H_{i-1}, M_i)$$

where f is the compression function.

Here,

$M_1, M_2, \dots, M_n$ = message blocks

$H_0 = IV$ = initial chaining value

$H_1, H_2, \dots, H_n$ = chaining variables

H_n = final hash value.

Length Extension Attack:

In Merkle-Damgård hashes, the final hash value contains the internal chaining state after processing the original message.

If an attacker knows $H(M)$

The attacker may be able to calculate:

$$H(M \parallel \text{padding} \parallel M')$$

without knowing the original message M .

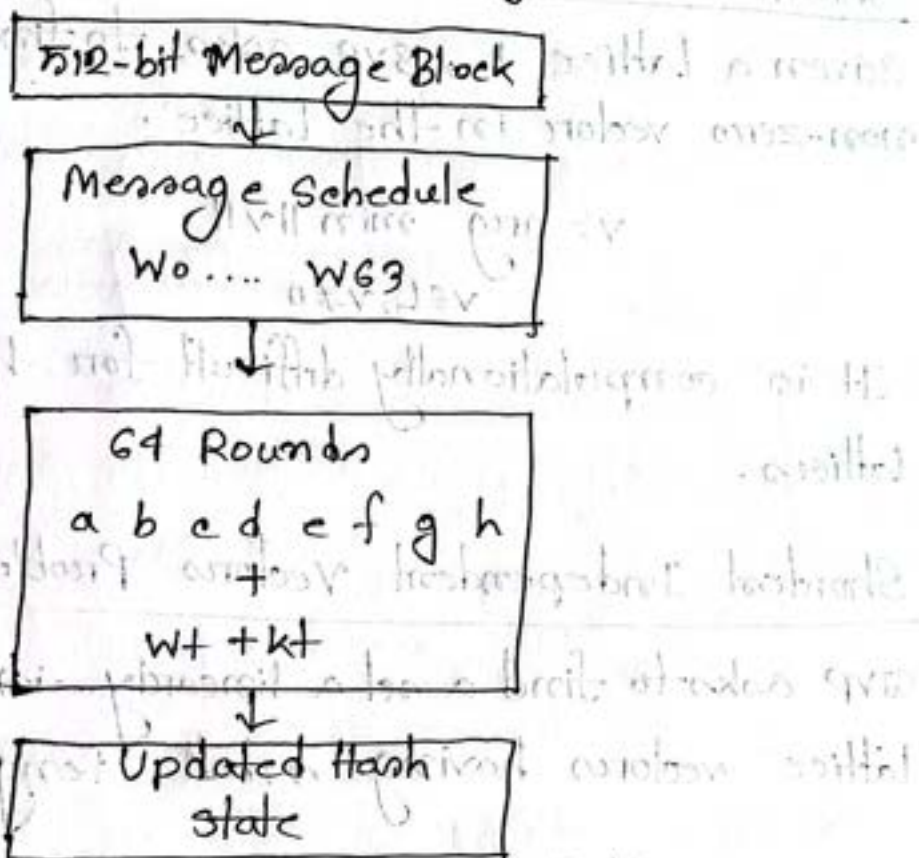
Therefore, if a system uses:

$\text{Hash}(\text{secret} \parallel \text{message})$ as a MAC, an attacker can potentially append extra data and create a valid hash.

(33)

SHA-256 Compression Function:

SHA-256 processes the message in 512-bit blocks.



The 512-bit block is divided into sixteen 32 bit words and expanded into: $W_0, W_1, \dots, W_{63}$

SHA-256 uses eight working variables:

a, b, c, d, e, f, g, h

The compression function performs 64 rounds. Each round uses the working variables, message word W_t and round constant k_t .

Finally SHA-256 produced a 256-bit hash.

(34)

SVP and SIVP:

Shortest Vector Problem (SVP):

Given a lattice L , SVP asks to find the shortest non-zero vector in the lattice.

$$v = \arg \min_{v \in L, v \neq 0} \|v\|$$

It is computationally difficult for high dimensional lattices.

Shortest Independent Vectors Problem (SIVP):

SIVP asks to find a set of linearly independent lattice vectors having small lengths.

$$v_1, v_2, \dots, v_n$$

must be linearly independent and as short as possible.

Quantum Resistance:

Shor's algorithm can efficiently solve integer factorization and discrete logarithm problems. However no efficient quantum algorithm is currently known for solving general high

dimensional SVP or SIVP.

Therefore, lattice problems are considered suitable foundations for Post-Quantum cryptography.

(35)

CRYSTALS - kyber and Module-LWE:

CRYSTALS - kyber is a lattice based Key Encapsulation Mechanism based on the Module-LWE problem.

The basic idea is $b = As + e$.

where,

A = public matrix

s = secret vector

e = small random noise

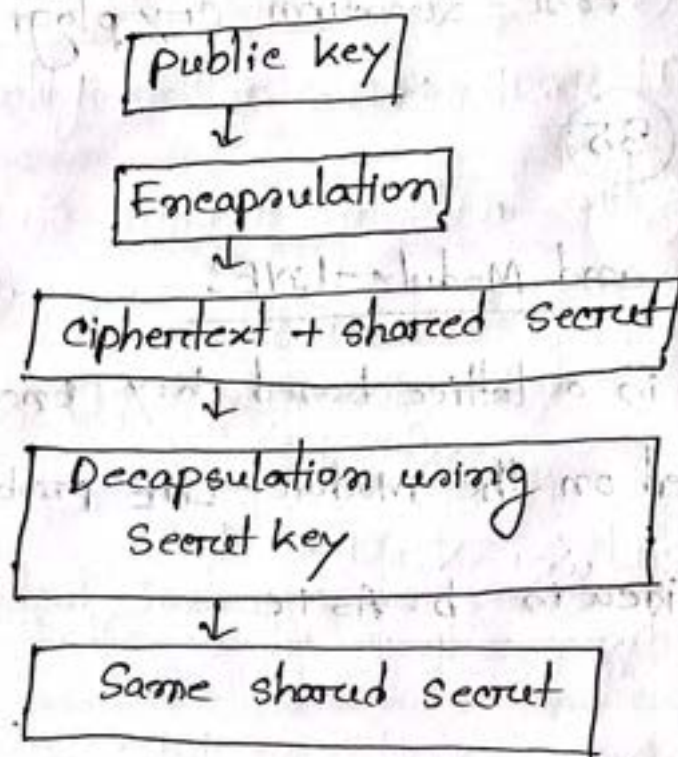
b = public value.

The noise e hides the relationship between the public value and the secret.

An attacker can see A and B , but recovering s is computationally difficult because of the

added noise.

The general process is:



Kyber's security is based on the hardness of Module LWE and related lattice problems.

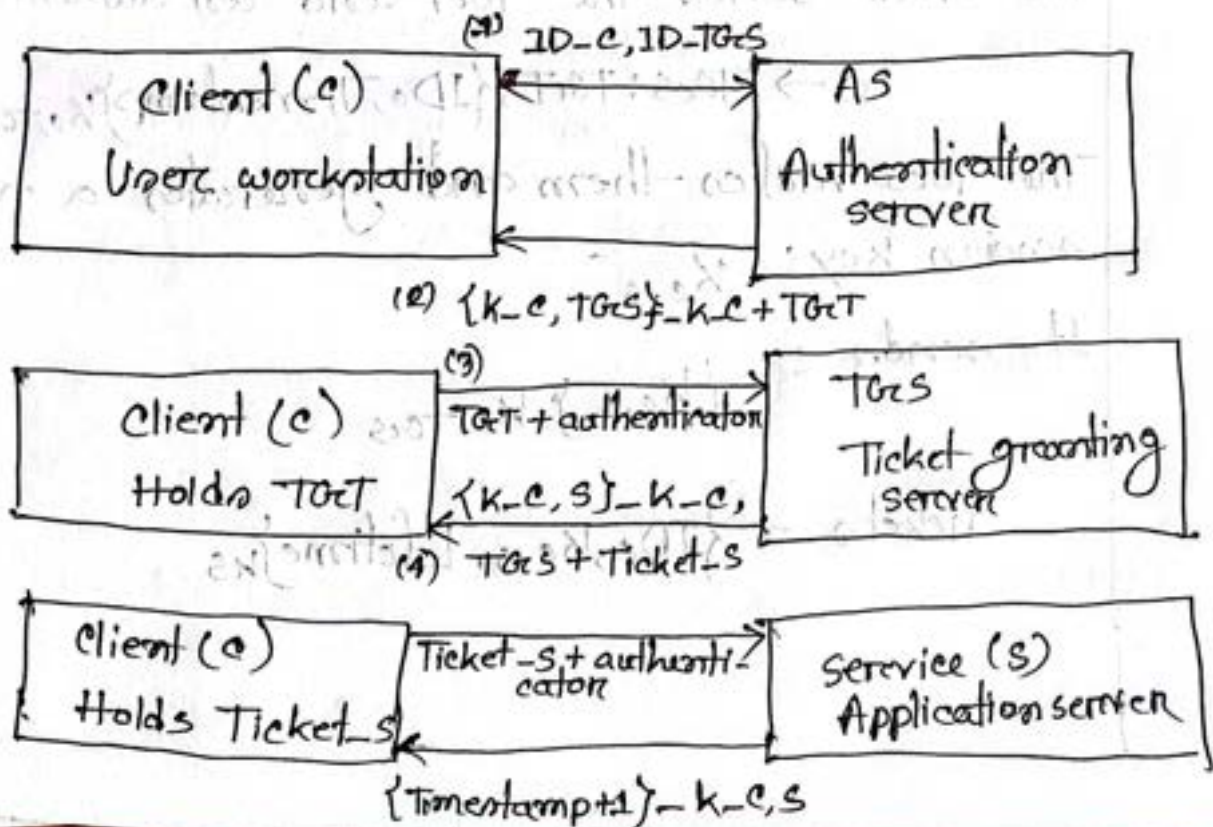
Shor's algorithm does not efficiently solve these lattice problems. Therefore Kyber is designed to resist both classical and known quantum attacks.

(36)

Diagram and explain the full Kerberos authentication protocol, covering the authentication. Server (AS) exchange, Ticket - Granting Server (TGS) exchange and client - Server (CS) exchange. Include how session keys and ticket lifetimes are established and why the TGT avoids repeated password transmission.

⇒ Kerberos is a network authentication protocol that provides secure authentication using tickets and session keys.

Protocol Diagram:



Step 1: Authentication Server (AS) exchange:

The client sends its identity and the TGS identity.
 $C \rightarrow AS: (ID_c, ID_{TGS})$

The AS verifies the client and generates a session key.

It sends $\{K_{c,TGS}\}_{K_c}$

$TGT = \{ID_c, K_{c,TGS}, Lifetime\}_{K_{TGS}}$

Here, K_c is the client's secret key.

Step 2: Ticket Granting Server (TGS) Exchange:

The client sends the TGT and an authenticator:

$C \rightarrow TGS: TGT, \{ID_c, Timestamp\}_{K_{c,TGS}}$

The TGS verifies them and generates a new session key: $K_{c,s}$

It sends: $\{K_{c,s}\}_{K_{TGS}}$

Tickets = $\{ID_c, K_{c,s}, Lifetime\}_{K_s}$

Step 3: Client - Server (CS) Exchange:

The client sends the service ticket and authentication:

$C \rightarrow S: \text{Ticket}_s, \{ID_c, \text{Timestamp}\}_{K_{c,s}}$

The server verifies them and replies:

$S \rightarrow C: \{\text{Timestamp}+1\}_{K_{c,s}}$

This provides mutual authentication.

Why does TGT avoid repeated password transmission?

$\Rightarrow$ The client password is used only during the initial authentication. After receiving the TGT, the client uses the TGT and $K_{c,s}$ to obtain service tickets. Therefore the password does not need to be transmitted repeatedly.

(38)

Explain why a plain hash-based MAC is vulnerable to a length extension attack, and derive how HMAC's nested construction $H(k \oplus \text{opad} \parallel H(k \oplus \text{ipad} \parallel M))$ eliminates this vulnerability.

⇒ A simple hash-based MAC can be defined as

$$\text{MAC} = H(k \parallel M)$$

where k is the secret key and M is the message.

Length Extension attack:

Some hash function based on the Merkle-Damgård construction are vulnerable to length extension.

If an attacker knows:

$$H(k \parallel M)$$

The attacker may be able to calculate:

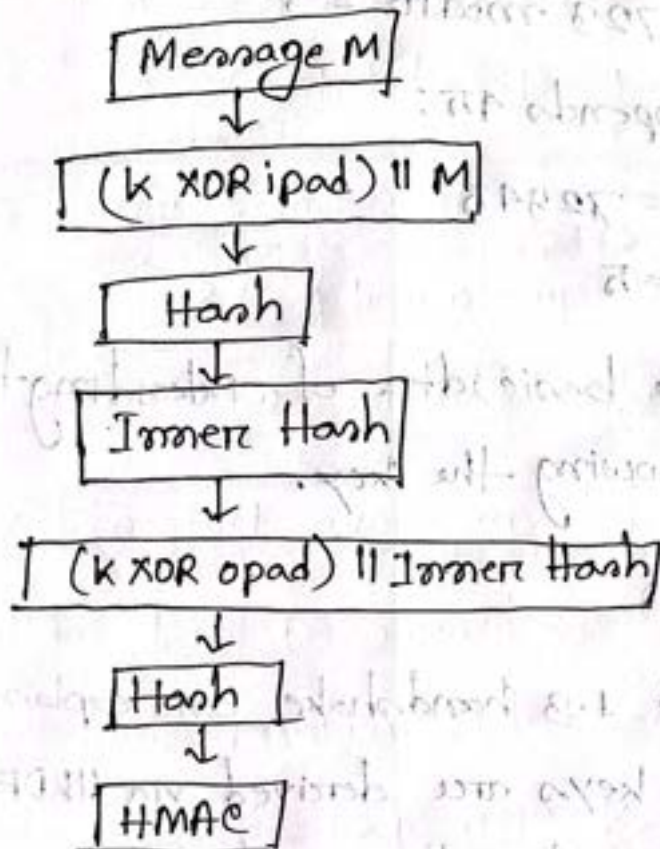
$H(k \parallel M \parallel \text{padding} \parallel M')$ without knowing the secret key k .

HMAC construction:

HMAC uses:

$$H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel M))$$

HMAC diagram:



Why HMAC prevents the attack:

In HMAC, the inner hash result is not directly used as the final MAC.

$$\text{Instead, Inner} = H((k \oplus \text{ipad}) || M)$$

$$\text{then, HMAC} = H((k \oplus \text{opad}) || \text{Inner})$$

Example: Suppose:

$$H(x) = x \bmod 10$$

$$\text{and } k = 7, M = 23$$

$$K_{111M} = 723$$

$$MAC = H(723) = 723 \bmod 10 = 3$$

If an attacker appends 45:

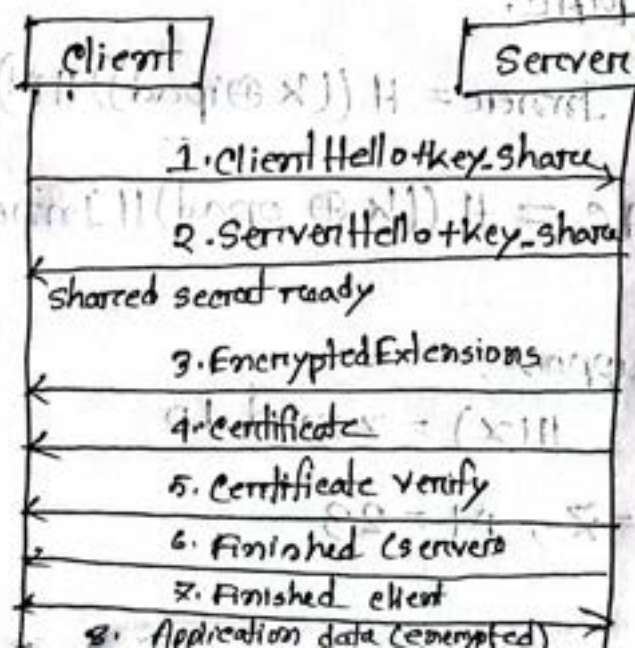
$$723 || 45 = 72345$$

$$H(72345) = 5$$

This illustrates the basic idea of extending the input without knowing the key.

(38)

Diagram the TLS 1.3 handshake and explain how the session keys are derived via HKDF from the (EC)DHE shared secret. Compare this against the TLS 1.2 handshake in terms of round trip count and forward secrecy.



Step 1:

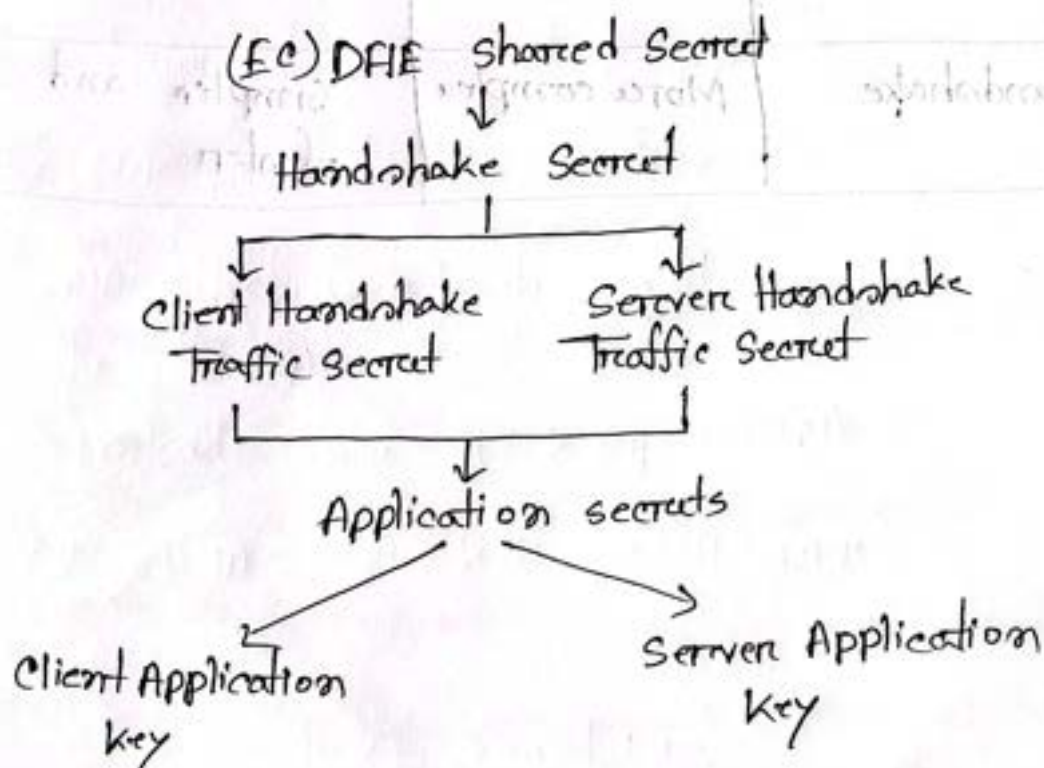
ClientHello: The client sends supported parameters and ephemeral key share: g^a

ServerHello: The server sends its key share: g^b

Both sides calculate the same Diffie-Hellman shared secret $Z = g^{ab}$

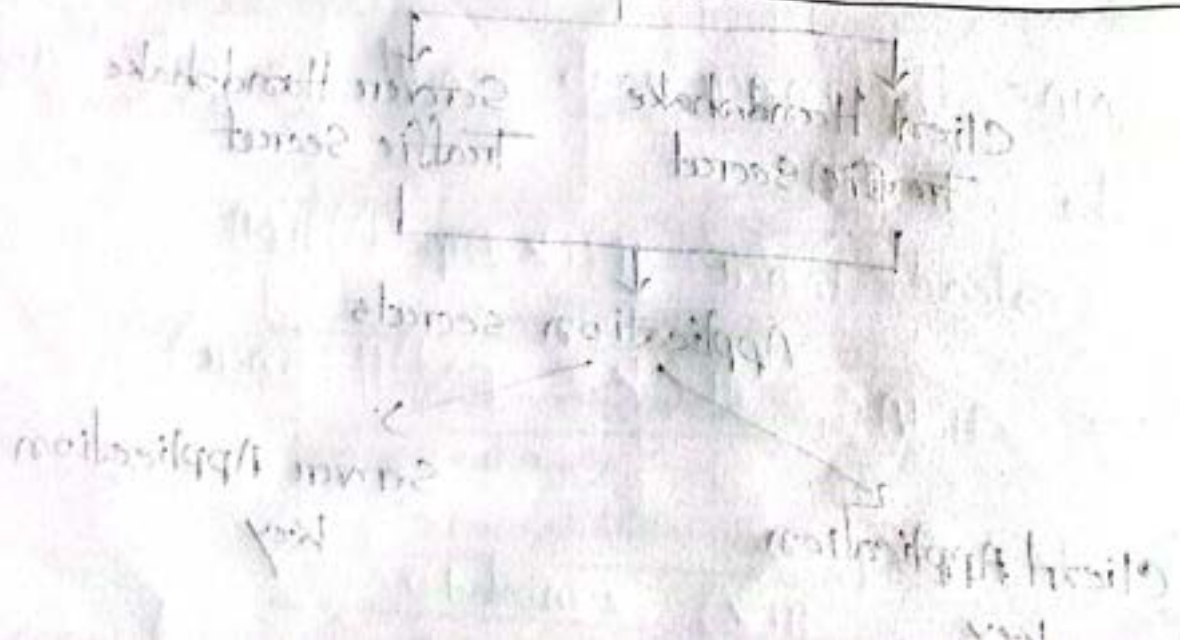
The shared secret is then used as input to the TLS 1.3 key schedule.

HKDF Key Derivation: TLS 1.3 uses HKDF to derive the session/traffic secrets.



Thus separate keys are generated for each communication direction.

Feature	TLS 1.2	TLS 1.3
Typical full handshake	2RTT	1RTT
Key exchange	RSA or DHE/ ECDHE	(EC)DHE
Forwarded Secrecy	Only with DHE/ ECDHE	provided by ephemeral (EC) DHE
Key derivation	PRF	HKDF
Handshake	More complex	Simpler and faster



(39)

IoT Bandwidth and Duty-cycle Budget:

Consider an IoT sensor network where each node sends: 1200 bytes once every minute.

There are: $24 \times 60 = 1440$ minutes/day

Therefore packets/day = 1440

Daily data = $1200 \times 1440 = 1,728,000$ bytes/day.

1kB = 1024 bytes.

$$\frac{1,728,000}{1024} = 1687.5 \text{ kB/day}$$

Duty-cycle Constraint:

Suppose the regulatory duty-cycle limit is 1%.

One day contains = $24 \times 60 \times 60 = 86400$ seconds

Maximum allowed airtime = 86400×0.01
= 864 second/day.

For 1440-transmissions = $\frac{864}{1440} = 0.6$ sec/packet

Therefore the packet occupies the channel for more than 0.6 seconds, the node would exceed a 1% duty-cycle limit.

Ascon vs AES-128 for Resource-Constrained IoT:

Ascon is a lightweight cryptographic algorithm selected by NIST for lightweight cryptography. AES-128 is a widely used 128-bit block cipher.

Feature	Ascon	AES-128
Design goal	Resource-constrained devices	General-purpose security
Key size	128-bit	128-bit
Memory	Designed for small devices	Can be implemented with low memory
Energy	Designed for low resource operation	Efficient, especially with hardware acceleration
IoT suitability	Highly suitable	Suitable
Hardware support	Lightweight implementations	Very widespread hardware support

Memory Footprint: Ascon is specially designed for devices with limited memory and processing power.

AES-128 can also be implemented with a small memory footprint.

Energy consumption: Aseon is designed to provide cryptographic security with low resource requirements. AES-128 can also be energy efficient when the device has dedicated AES hardware acceleration.

Side Channel Resistance: Neither algorithm is automatically protected against side channel attacks.

Both require techniques such as:

- i) Masking
- ii) Constant time implementation
- iii) Avoiding secret dependent memory access
- iv) Fault detection.

For a battery powered sensor with

RAM $\leq 10\text{KB}$

Aseon is suitable choice because it is specifically designed for lightweight and resource constrained devices.